

# AI Sandbox — Product Requirements Document

Cloud-Based Ephemeral Sandbox Platform for AI Agents | Version 1.0 | Phase 3 baseline

## 1. Project Overview

Cloud-Based Ephemeral Sandbox Platform for AI Agents is a cloud-oriented backend platform for safely executing untrusted or AI-generated code in isolated, temporary environments.

The platform exposes an API that AI agents or other clients can call to create logical sandboxes, execute code, inspect execution results, and manage sandbox lifecycle.

## 2. Problem Statement

Running AI-generated code directly on a host machine creates security and reliability risks including filesystem access, excessive resource consumption, process creation, network access, and interference with other workloads.

The project addresses this by combining an API layer, persistent sandbox/job metadata, and hardened ephemeral Docker execution.

## 3. Goals

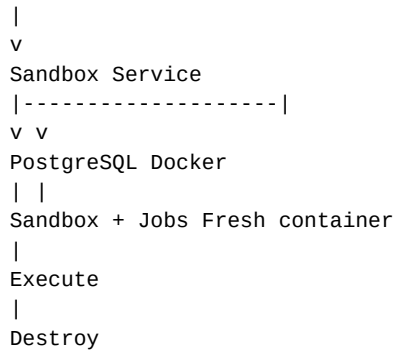
- Secure execution of untrusted code.
- Ephemeral containers for every execution.
- Simple HTTP API suitable for agent tool calls.
- CPU, memory, PID, timeout, and output limits.
- Persistent sandbox and execution-job lifecycle.
- Cloud deployability, initially targeting infrastructure such as AWS EC2.

## 4. Non-Goals

- Kubernetes orchestration in the initial system.
- Redis/Celery/Kafka unless a later phase explicitly requires them.
- Multi-region infrastructure.
- Unrestricted network access.
- Arbitrary Docker image selection.
- Persistent writable filesystem across executions.

## 5. Core Architecture

```
AI Agent
|
v
FastAPI
```



A Sandbox is a persistent logical resource in PostgreSQL. It is not the Docker container. Each ExecutionJob receives a fresh ephemeral container that is removed after execution.

## 6. Sandbox and Execution Semantics

One Sandbox can have many ExecutionJobs. The current implementation does not preserve writable filesystem state between executions.

Sandbox TTL controls the lifetime of the logical sandbox. Execution timeout controls the maximum duration of one execution. An execution can time out while its sandbox remains usable.

## 7. Functional Requirements

- GET /health for process health.
- GET /ready for infrastructure readiness.
- POST /api/v1/sandboxes to create a sandbox.
- GET /api/v1/sandboxes/{sandbox\_id} to inspect it.
- DELETE /api/v1/sandboxes/{sandbox\_id} to destroy it.
- Execute code within a sandbox and persist the result.
- Return bounded stdout/stderr, exit code, status, duration, and error information.

## 8. Lifecycle Requirements

Sandbox lifecycle includes running, expired, destroying, and destroyed states. Expired sandboxes are unusable for new executions but remain inspectable until cleanup/deletion.

Execution jobs use explicit states such as pending, running, completed, timed\_out, and failed. Invalid transitions must be rejected.

## 9. Security Requirements

- network\_mode=none.
- Read-only root filesystem.
- Non-root user 10001:10001.
- cap\_drop=ALL.
- no-new-privileges.
- Memory limit 256 MB.

- CPU allowance approximately 0.5 core.
- PID limit 32.
- 32 MB /tmp with noexec,nosuid,nodev.
- No host filesystem mounts.
- No Docker socket exposure.
- No arbitrary image selection.
- Bounded stdout/stderr.
- Mandatory cleanup on completion, failure, and timeout.
- Only platform-owned containers may be cleaned up.

## 10. Persistence

PostgreSQL stores Sandbox and ExecutionJob metadata. SQLAlchemy 2.x is used for persistence and Alembic for migrations.

## 11. Technology Stack

- Python
- FastAPI
- Pydantic
- SQLAlchemy 2.x
- Alembic
- PostgreSQL
- Docker + Docker Python SDK
- pytest
- Ruff

## 12. Testing and Definition of Done

- Unit, integration, API, and security tests.
- Existing tests must remain passing after future phases.
- Ruff must pass.
- Migrations must be valid where applicable.
- No secrets may be committed.
- Security controls must remain intact.
- Container cleanup must be verified.
- Unrelated Docker resources must remain untouched.

## 13. Project Phases

Phase 1 — Secure ephemeral Docker execution: COMPLETE.

Phase 2 — FastAPI API layer: COMPLETE.

Phase 3 — PostgreSQL persistence and sandbox/job lifecycle: COMPLETE and verified.

Phase 4 onward — Continue toward AI-agent integration, cloud deployment, observability, and scalability according to the phase-specific prompts.

## **14. Security Invariants**

Future development must not casually weaken isolation, resource limits, cleanup, non-root execution, network restrictions, or secret handling. Any deliberate security change must be documented and tested.